

《基于Xdp实现的udp负载均衡器》

使用技术栈: rust、xdp、Linux shell

团队成员与分工

成员: 陈科年 (24336015)

分工: 负责项目的开发、测试、文档编写以及后续拓展维护

项目背景与目标

1. 项目介绍: 该项目是在Linux环境下对于网络转发工作(这里只有udp), 使用基于udp的负载均衡器, 绕开使用完整的linux内核协议栈处理客户端发送的数据包, 使用xdp处理udp数据包后均衡转发(使用哈希实现) 给后端服务器(项目中使用nginx服务作为模拟)
2. 为什么选择该题目: 主要是学习xdp技术在rust下的开发方式(xdp目前在rust的技术栈并没有那么成熟, 并学习udp数据包报文结构以及协议栈)
3. 实现功能: 对于udp数据包报文的Full NAT转发, 转发中使用哈希动态计算选择后端服务器, 实现流量均衡, 并且负载均衡器维护后端的健康检查
4. 项目解决了什么问题: 在保证后端会话的一致的前提下, 降低转发的流量延迟

系统设计与实现思路

1. 核心系统架构: 控制面与数据面解耦

系统采用标准的 **Rust Workspace** (工作区) 结构, 将网络包处理与业务逻辑进行隔离。

- 数据面 (**Data Plane - udp-lb-ebpf**):
 - 运行环境: 挂载于 Linux 网卡驱动层(XDP)的 eBPF 虚拟机中。
 - 核心职责: 拦截网卡入栈流量, 在数据包进入 Linux 内核协议栈(Skb 阶段)之前, 直接对 L2/L3/L4 头部进行解析、哈希寻址、NAT 转换及包重定向(**XDP_TX** 或 **XDP_PASS**)。
- 控制面 (**Control Plane - udp-lb**):
 - 运行环境: 宿主机用户空间, 基于 Tokio 异步运行时。
 - 核心职责: 解析 YAML 配置文件、下发 eBPF 字节码、维护一致性哈希环、异步健康检查(心跳探测)、以及连接表老化回收(GC)。
 - 通信桥梁: 通过 eBPF Maps(**Array** 和 **HashMap**)与数据面进行零拷贝的内存共享与状态同步。

2. 数据面 (eBPF) 核心实现机制

- 流量分发引擎 (**main.rs**):
 - 使用前置边界检查器(**ptr_at_mut**)依次剥离 EthHdr、Ipv4Hdr 和 UdpHdr。
 - 根据目标 IP 将流量严格拆分为: 正向入站流量(访问 **VIP**)和反向回包流量(访问 **LIP**)。
- **FullNAT** 路由重写 (**fwd.rs** & **rev.rs**):

- **SNAT + DNAT**: 进站时, 将源 IP 替换为 LIP, 目的 IP 替换为后端 RS 的 IP; 回包时反之。
- 防 **MAC 欺骗**: 每次 **XDP_TX** 弹回网桥前, 强制将数据包的源 MAC 地址覆写为 LB 网卡自身的 MAC, 避免虚拟网桥 MAC 学习表混乱导致流量黑洞。
- 高速连接追踪 (**Conntrack**):
 - 基于 **HashMap** 构建双向连接追踪表 (**CONNTRACK_FORWARD** 和 **CONNTRASE_REVERSE**)。
 - 采用 **Jenkins Hash** 算法结合客户端 IP、端口与动态种子, 将新会话映射到 1024 槽位的一致性哈希环中。
 - 通过 **get_ptr_mut()** 返回裸指针并在 **unsafe** 块中就地更新 **last_active** 时间戳, 实现 O(1) 复杂度的缓存命中与状态刷新。

3. 控制面 (Tokio) 核心实现机制

- 动态哈希环下发 (**ring.rs**):
 - 在用户态解析后端节点 (RS1, RS2), 预计算并填充哈希槽位到 **RING_LOOKUP_TABLE** Map 中。内核仅需执行 **hash % ring_size** 的极简查表操作。
- 守护协程与垃圾回收 (**conntrack.rs & health.rs**):
 - **UDP 心跳探针**: 利用 **tokio::net::UdpSocket** 每 5 秒向所有后端发起异步探测, 具备毫秒级超时熔断机制。
 - 双向 **GC 回收器**: 每 10 秒轮询一次正向会话表。对比当前 **CLOCK_MONOTONIC** 时钟与会话的 **last_active**, 若超时 (如 30 秒无流量), 则基于 RS 信息反推并擦除双向 Map 记录, 防止 UDP 僵尸会话引起的内存泄露和哈希碰撞。

实验结果

1. 本机测试的网络环境 (使用 namespace 网络隔离, 网桥连接模拟)
可以使用项目中的 `./setup_env.sh` 脚本得到 (先执行 `./build.sh` 脚本构建项目)

```
Client 节点  -> Namespace: ns-client | IP: 10.0.0.5
LB 负载均衡  -> Namespace: ns-lb   | VIP: 10.0.0.1, LIP: 10.0.0.254
RS1 服务器   -> Namespace: ns-rs1  | IP: 192.168.1.10 (Python UDP 8080 监听中)
RS2 服务器   -> Namespace: ns-rs2  | IP: 192.168.1.11 (Python UDP 8080 监听中)
```

2. 连通性测试结果
可以通过项目中的 `./test_lb.sh` 脚本得到 (需要先执行 `./build.sh` 以及 `./test_lb.sh`)

--> 开始从 ns-client 发起 UDP 探测请求 (共 8 次)...

```
第 1 次请求命中 -> [RS2] 192.168.1.11
第 2 次请求命中 -> [RS2] 192.168.1.11
第 3 次请求命中 -> [RS2] 192.168.1.11
第 4 次请求命中 -> [RS1] 192.168.1.10
第 5 次请求命中 -> [RS2] 192.168.1.11
第 6 次请求命中 -> [RS1] 192.168.1.10
第 7 次请求命中 -> [RS1] 192.168.1.10
第 8 次请求命中 -> [RS1] 192.168.1.10
```

测试数据统计结论:

后端 Real Server 1 命中次数: 4

后端 Real Server 2 命中次数: 4

3. Latency测试结果

可以通过项目中的./benchmark_latency.sh脚本获得(需要先执行./build.sh脚本构建项目)

[对照组] 正在测试: 传统 Linux 内核协议栈转发 (Target: RS1 192.168.1.10)...

[测试组] 正在测试: eBPF XDP 极速无锁转发 (Target: VIP 10.0.0.1)...

延迟基准测试结果 (Round Trip Time)

转发路径	平均延迟 (ms)	最小延迟 (ms)	最大延迟 (ms)	P99 延迟 (ms)	成功率
1. Linux Kernel	0.0236	0.0153	1.3075	0.0746	1000/1000
2. eBPF XDP (VIP)	0.0198	0.0148	0.3404	0.0674	1000/1000

性能结论: XDP 负载均衡器使平均网络延迟降低了 **16.10%**!

抖动分析: Linux 内核的 P99 抖动为 0.051ms, XDP 抖动为 0.0476ms.